

MODULE 11 · PART 3 - CTES AND RECURSION

Recursive CTEs: The Mechanism

The only construct in SQL that can refer to itself. And it is not recursion.

PART 3 of 8 MODULE 11 of 26 RUNS about 34 min

The last resort, and sometimes the only one

- ◆ **Recursion is the most expensive construct in this course.** Every pass is a fresh join, and there is no way to parallelise across passes.
- ◆ **It is also the only one that can fail to terminate.** Nothing else in SQL can run forever on finite data.
- ◆ **And for a hierarchy of unknown depth there is no alternative,** short of storing the answer or writing the same join n times.
- ◆ **So the question is never whether recursion is expensive.** It is whether the depth is genuinely unknown.

THE TAKEAWAY

If you know the tree is six deep, six joins will beat a recursive walk. If you do not know, or it changes, recursion is the only thing that survives.

The same tree, without recursion

```
1 SELECT c1.category_name AS level_1,  
2        c2.category_name AS level_2,  
3        c3.category_name AS level_3,  
4        c4.category_name AS level_4  
5 FROM dim_category c1  
6  LEFT JOIN dim_category c2 ON c2.parent_key = c1.category_key  
7  LEFT JOIN dim_category c3 ON c3.parent_key = c2.category_key  
8  LEFT JOIN dim_category c4 ON c4.parent_key = c3.category_key  
9 WHERE c1.parent_key IS NULL;
```

8 One join per level. Adding a fifth level means editing the query, and nobody will notice it is needed.

THE TAKEAWAY

Faster, parallelisable, and wrong the day somebody adds a level. This is what recursion replaced, and it is still the right answer when the depth is fixed by design.

Three ways to store a hierarchy

- ◆ **Adjacency list.** Each row holds its parent. Trivial to maintain, and every read is a recursive walk.
- ◆ **Path enumeration.** Each row stores its full path as a string or array. Reads are cheap, and a move rewrites a subtree.
- ◆ **Closure table.** One row per ancestor-descendant pair. Reads are a join, and writes are proportional to depth.
- ◆ **Kestrel uses an adjacency list**, like nearly everybody, because it is what an operational system naturally produces.

THE TAKEAWAY

The storage choice decides whether you need recursion at all. An adjacency list makes every read a walk, and that is the trade it was making.

Which storage, for which problem

QUESTION	ADJACENCY LIST, AND CLOSURE TABLE
Who is my parent?	One lookup. Or one lookup.
Who are all my descendants?	A recursive walk. Or one indexed join.
How deep am I?	A recursive walk. Or a stored column.
Move a subtree	One update. Or rewrite every pair beneath it.
Add a leaf	One insert. Or one insert per ancestor.

THE TAKEAWAY

A closure table converts read cost into write cost. For a tree that changes twice a month and is read constantly, that is an obviously good trade.

A recursive CTE computes a result, then feeds that result **back into itself**, until nothing new comes out.

What it is for

- ◆ **Hierarchies of unknown depth.** A category tree, an org chart, a bill of materials, a folder structure.
- ◆ **Chains and paths.** Follow this reference to the next one, and keep going until there is no next one.
- ◆ **Generating rows from nothing.** A series of numbers, a spine of dates, a set of intervals.
- ◆ **Calculations that carry state forward**, where each step needs the previous step's answer. Windows cannot do this.

THE TAKEAWAY

The last one is the boundary. A running total is a window. A compounding balance, where each step's input is the last step's output, is recursion.

The smallest possible example

```
1 WITH RECURSIVE counter AS (  
2   SELECT 1 AS n           -- the anchor  
3   UNION ALL  
4   SELECT n + 1           -- the recursive term  
5   FROM counter  
6   WHERE n < 10           -- the stopping condition  
7 )  
8 SELECT n FROM counter;
```

- 2 The anchor runs once. It produces the starting rows and it fixes the column types for the whole result.
- 5 The self-reference. This is what makes the block recursive, and there may be exactly one of them.
- 6 The stopping condition. Without it, this query never terminates and nothing in the syntax warns you.

THE TAKEAWAY

Three parts, and only the middle one is compulsory syntax. The stopping condition is an ordinary WHERE clause that you have to remember to write.

The anatomy

PART	WHAT IT DOES
WITH RECURSIVE	Declares that a block may refer to itself. Required even for one block among many.
The anchor term	Runs once. Produces the seed rows, and fixes every output column's type.
UNION or UNION ALL	Combines each generation with the result. The choice matters.
The recursive term	Refers to the block by name. Sees only the previous generation.
The stopping condition	An ordinary predicate. Nothing enforces its existence.

THE TAKEAWAY

The fourth row is the one people get wrong, and most of this module is about it.

It is **iteration**, not recursion. PostgreSQL's own documentation says so.

The algorithm, in five steps

01

Seed

Run the anchor.
Put its rows in
the result, and
also in a working
table.

02

Substitute

Run the recursive
term, with the
self-reference
replaced by the
working table.

03

Collect

Put the new rows
in the result, and
in an
intermediate
table.

04

Swap

Replace the
working table
with the
intermediate
table, and empty
the intermediate.

05

Repeat

Go to step two.
Stop when the
working table
comes back
empty.

THE TAKEAWAY

Two scratch tables and a loop. Every restriction on recursive CTEs, and every surprise in them, follows from this algorithm rather than from theory.

The recursive term sees **only the previous generation**. Not the result so far.

The working table, traced

PASS	WORKING TABLE CONTENTS
anchor	{1}
1	{2}
2	{3}
...	...
9	{10}
10	{ } and the loop ends

THE TAKEAWAY

One row at a time, for the counter example. The result accumulates to ten rows and the working table never holds more than one of them.

Reading a recursive CTE out loud

PART	SAID
<code>WITH RECURSIVE walk AS (</code>	let walk be built up in stages
<code>SELECT ... WHERE parent_key IS NULL</code>	start with the roots
<code>UNION ALL</code>	and keep adding
<code>SELECT ... FROM dim_category c JOIN walk w</code>	the children of whatever we found last time
<code>ON c.parent_key = w.category_key</code>	matched to their parent
<code>) SELECT * FROM walk;</code>	until there are no more children

THE TAKEAWAY

'Whatever we found last time' is the phrase to keep. Not everything found so far. That distinction is the whole mechanism in four words.

What follows from that

- ◆ **You cannot check whether a node was already visited.** The rows that would tell you are not in the working table.
- ◆ **You cannot aggregate over the self-reference.** There is nothing to aggregate except one generation, and engines reject it outright.
- ◆ **So carry it as a column.** Depth, path, running value, visited set. All of them ride on the rows.
- ◆ **That single technique is most of recursive SQL.** If a step needs to know something, put it on the row.

THE TAKEAWAY

Every recursive query is a decision about what each row carries. The traversal is the easy part, and the columns are where the thinking is.

Carrying state forward

```
1 WITH RECURSIVE walk AS (  
2   SELECT category_key, parent_key, 1 AS depth,  
3         ARRAY[category_key] AS path  
4   FROM dim_category WHERE parent_key IS NULL  
5   UNION ALL  
6   SELECT c.category_key, c.parent_key, w.depth + 1,  
7         w.path || c.category_key  
8   FROM dim_category c  
9   JOIN walk w ON c.parent_key = w.category_key)  
10 SELECT * FROM walk;
```

- 6 Depth is carried, not computed. The previous row knew its depth, so this row adds one.
- 7 The path is appended to. Each row carries the route it took to get here.

THE TAKEAWAY

Neither column can be derived from the current row alone. Both exist only because the previous generation carried them.

Why depth and path are worth carrying

- ◆ **Depth gives you a stopping condition**, an indent for display, and a way to cap a runaway query.
- ◆ **Path gives you cycle detection**, because a node already in the path is a node you have seen.
Module twelve.
- ◆ **Path also gives you ordering**, because sorting by it produces depth-first order rather than breadth-first.
- ◆ **Both cost almost nothing**, and both are nearly impossible to add later without rewriting the query.

THE TAKEAWAY

Write depth and path into every hierarchy walk, whether or not you need them yet. They are the two columns you will wish you had.

The algorithm produces **breadth-first** order. Whole generations at a time.

Depth-first, from a path column

```
1 SELECT repeat(' ', depth - 1) || category_name AS tree,  
2         depth, path  
3     FROM walk  
4     ORDER BY path;
```

- 4 Sorting by the path array gives depth-first order. Each branch is followed to its end before the next begins.
- 1 And depth gives the indent, so the output reads as a tree rather than as a list.

THE TAKEAWAY

Two columns carried through the recursion turn an unordered set of nodes into something a person can read. Neither is available any other way.

An analyst walks `dim_category` recursively, carrying a path built by concatenating `category_name` with a separator. The anchor selects `category_name` directly, which is declared `VARCHAR(60)`.

The query fails at the fourth level with a value-too-long error, and only on branches whose category names are long. The tree is six levels deep at most, and the failure is reproducible.

What is the cause?

- ☐ A The path should be an array of keys rather than a concatenated string, which has no length limit.
- ☐ B `dim_category` contains a cycle, so the path grows without bound until it exceeds the column width.
- ☐ C The anchor fixes the output column types, so the path column is `VARCHAR(60)` for the whole result. Cast it in the anchor.
- ☐ D The recursive term needs an explicit depth cap, since nothing else stops the path growing.

◆ PAUSE AND DECIDE

What is the cause?

A Use an array of keys

Good advice and a different fix. It works by changing the anchor's type to an array, which is the same lesson wearing different clothes.

C The anchor fixes the types

Every output column takes its type from the anchor term. Casting category_name to text in the anchor makes the path unbounded.

B A cycle in the data

There is one, and it is not this. A cycle fails at unbounded depth on the affected branch, not at level four on branches with long names.

D Add a depth cap

Stops the query before it fails rather than fixing it. The tree is six deep and the analyst needs all six levels.

THE SIGNAL

"Only on branches whose category names are long." A failure correlated with string length, at a fixed depth, is a width problem. A cycle correlates with the branch, not the names.

Descendants of one node

```
1 WITH RECURSIVE sub AS (  
2     SELECT category_key, parent_key, 1 AS depth  
3     FROM dim_category  
4     WHERE category_key = 3310          -- one node, not all roots  
5     UNION ALL  
6     SELECT c.category_key, c.parent_key, s.depth + 1  
7     FROM dim_category c  
8     JOIN sub s ON c.parent_key = s.category_key)  
9 SELECT * FROM sub;
```

- 4 The anchor is one row instead of every root. The walk now touches one subtree rather than the whole tree.

THE TAKEAWAY

Filtering the anchor is the single largest optimisation available to a recursive query, and it is one line. Everything downstream is proportional to the seed.

Filter the anchor, not the result

- ◆ **Everything is proportional to the seed.** Halving the anchor roughly halves every pass after it.
- ◆ **Filtering the outer query does not help.** The walk has already happened by the time the outer WHERE runs.
- ◆ **And filtering inside the recursive term prunes branches,** which is the second-largest lever after the anchor.
- ◆ **Module one, again.** The outer query is a later step, and a later step cannot reduce an earlier one's work.

THE TAKEAWAY

A recursive block that walks the whole tree and then filters to one branch has done all the work and thrown it away. The filter belongs in the anchor.

Upwards instead of downwards

```
1 WITH RECURSIVE up AS (  
2     SELECT category_key, parent_key, 1 AS levels_up  
3     FROM dim_category WHERE category_key = 88412  
4     UNION ALL  
5     SELECT p.category_key, p.parent_key, u.levels_up + 1  
6     FROM dim_category p  
7     JOIN up u ON u.parent_key = p.category_key)  
8 SELECT * FROM up;
```

- 7 The join is reversed. Downwards matches c.parent_key to w.category_key; upwards matches u.parent_key to p.category_key.

THE TAKEAWAY

One line separates ancestors from descendants. Everything else about the query is identical, which is why the direction is so easy to get backwards.

Up and down behave differently

- ◆ **Downwards can fan out.** A node with forty children produces forty rows in the next pass, and that compounds.
- ◆ **Upwards cannot.** Each node has one parent, so the walk is a single chain and its length is the depth.
- ◆ **So an upward walk is cheap and bounded,** and it terminates at the root without needing a stopping condition.
- ◆ **Unless the data has a cycle,** in which case upwards loops exactly as readily as downwards does.

THE TAKEAWAY

Upward walks feel safe and are not. A cycle makes a single chain into a circle, and a chain with no branching goes round it very efficiently indeed.

A bill of materials

```
1 WITH RECURSIVE bom AS (  
2   SELECT component_key, parent_key, qty, 1 AS depth  
3     FROM dim_meal_kit_component WHERE parent_key = 7781  
4   UNION ALL  
5   SELECT c.component_key, c.parent_key,  
6         b.qty * c.qty,          -- quantities multiply  
7         b.depth + 1  
8     FROM dim_meal_kit_component c  
9     JOIN bom b ON c.parent_key = b.component_key)  
10 SELECT component_key, SUM(qty) AS total_qty  
11 FROM bom GROUP BY component_key;
```

- 6 The carried quantity is multiplied, not added. Two of a sub-assembly containing three of a part is six.
- 10 And the totals are summed in the outer query, because a part can appear in several sub-assemblies.

THE TAKEAWAY

A bill of materials is a hierarchy where the carried value compounds. That multiplication is why this cannot be a join, however many levels you write.

What the carried column can do

- ◆ **Count, with depth plus one.** The simplest and the most useful, because it also caps the query.
- ◆ **Accumulate, with a path array or string.** For ordering, for display, and for cycle detection.
- ◆ **Multiply, as in a bill of materials.** Quantities compound down a tree rather than adding.
- ◆ **Or carry anything from the seed unchanged,** so every descendant knows which root it belongs to.

THE TAKEAWAY

The last one is quietly the most used. Carrying the root key down means a single walk can resolve every node's top-level category in one pass.

UNION removes duplicates against **every row found so far**, not just within one pass.

The two, compared

BEHAVIOUR	UNION ALL, AND UNION
Duplicate rows	Kept. Or discarded, against every row already in the result.
Cost	Nothing extra. Or a comparison of every new row against the whole result.
Loop on cyclic data	Runs forever. Or terminates, if the repeated rows are genuinely identical.
With a depth or path column	Runs forever. Or runs forever, because no two rows are ever identical.

THE TAKEAWAY

The last row is the trap. Adding the depth column that makes a query debuggable is exactly what disables UNION's accidental loop protection.

Why UNION is not loop protection

- ◆ **It compares whole rows.** Two visits to the same node are only duplicates if every carried column matches too.
- ◆ **A depth column makes every revisit unique,** so a cycle produces depth 4, 5, 6, 7 for ever and none of them is a duplicate.
- ◆ **A path column does the same, more so,** because the path is different on every lap by construction.
- ◆ **So use UNION ALL and handle cycles deliberately,** which is module twelve. Do not rely on a side effect.

THE TAKEAWAY

The two columns that make a recursive query understandable are the two that break UNION's incidental protection. That is not a coincidence, and it is scenario two.

A team walks `dim_category` with `UNION` rather than `UNION ALL`, on the stated grounds that `UNION` protects against loops. The block carries a depth column and a path string for debugging.

The query runs for ninety minutes and then fails when the cluster runs out of disk. A colleague points out that `UNION` is deduplicating, so the query cannot be producing duplicate rows.

Why does it not terminate?

- ☐ A `UNION` discards only exactly-duplicate rows, and the depth and path columns make every revisit unique.
- ☐ B `UNION` deduplicates only within a single pass, not against the rows found in earlier passes.
- ☐ C `UNION` is not permitted in a recursive CTE and is silently treated as `UNION ALL` by the engine.
- ☐ D The loop is in `dim_store`, not `dim_category`, so this query is not the one that needs fixing.

◆ PAUSE AND DECIDE

Why does it not terminate?

A The carried columns break it

The colleague is right that no duplicates are produced. That is the problem. Depth 4 and depth 5 on the same node are different rows.

C UNION is ignored here

It is permitted and it is honoured. The deduplication is running, at real cost, and finding nothing to remove.

B Only within one pass

A precise-sounding claim that is false. The documentation is explicit that UNION discards rows duplicating any previous result row.

D Wrong table

Both contain a loop. dim_category has one from a 2024 merchandising correction, and the employee chart has one from a TUPE load.

THE SIGNAL

"The query cannot be producing duplicate rows." Correct, and it is the diagnosis rather than the defence. Nothing is duplicated because every row carries a counter that keeps rising.

What the recursive term cannot contain

RESTRICTION	WHY, GIVEN THE ALGORITHM
More than one self-reference	The working table is substituted once.
An aggregate over the self-reference	It would aggregate one generation only.
The self-reference inside a subquery	Substitution happens at one place in the plan.
Self-reference on an outer join's null side	It would manufacture rows that never end.
ORDER BY or LIMIT on the recursive term	Generations are unordered sets.

THE TAKEAWAY

Every restriction follows from two scratch tables and a loop. None is arbitrary, and none needs memorising separately.

Dialect differences worth knowing

- ◆ **SQL Server does not use the RECURSIVE keyword at all.** A CTE that refers to itself simply is recursive.
- ◆ **And it caps recursion at 100 levels by default**, raised with an `OPTION (MAXRECURSION n)` hint, or removed with zero.
- ◆ **MySQL caps it too**, with a configurable depth limit, and it is the only engine that allows `LIMIT` in the recursive term.
- ◆ **Postgres, Redshift and Snowflake have no default cap.** A runaway query runs until something else stops it.

THE TAKEAWAY

SQL Server's cap is a safety rail that also breaks legitimate deep hierarchies. The engines without one will let a cycle consume the cluster.

Two clauses the standard added

- ◆ **SEARCH DEPTH FIRST BY key SET ordercol** builds the path column for you, and you order the outer query by it.
- ◆ **SEARCH BREADTH FIRST BY key SET ordercol** does the same for level-by-level order, which is the natural order anyway.
- ◆ **CYCLE key SET is _cycle USING path** builds the visited-set array and stops the walk. That is module twelve.
- ◆ **PostgreSQL 14 and later, and not much else.** Redshift and Snowflake have neither, so the manual version is the portable one.

THE TAKEAWAY

Both clauses generate exactly the columns this module has been carrying by hand. Knowing the manual form is what lets you read the generated one.

SEARCH, and what it generates

```
1 WITH RECURSIVE walk AS (  
2     SELECT category_key, parent_key FROM dim_category  
3     WHERE parent_key IS NULL  
4     UNION ALL  
5     SELECT c.category_key, c.parent_key  
6     FROM dim_category c  
7     JOIN walk w ON c.parent_key = w.category_key  
8 ) SEARCH DEPTH FIRST BY category_key SET ord  
9 SELECT * FROM walk ORDER BY ord;
```

8 One clause replaces the hand-built path array. The column ord holds it, and you still have to order by it yourself.

THE TAKEAWAY

It does not sort the result. It creates the column that lets you sort the result, which is the same distinction as module two's ordering inside a CTE.

Whether to use it

- ◆ **Shorter, and self-documenting** on an engine that has it. The intent is a keyword rather than an array expression.
- ◆ **Not portable.** A query using it will not run on Redshift, Snowflake, SQL Server or MySQL.
- ◆ **And it hides the mechanism**, so somebody who has only ever used it cannot debug a walk that misbehaves.
- ◆ **Learn the manual form first.** The clause is then obvious, and the manual form is what you will actually ship.

THE TAKEAWAY

This is the same judgement as QUALIFY in module eight. The convenient form is worth using where it exists, and the portable form is worth being able to write.

Generating rows from nothing. A recursive CTE can produce data that **no table contains.**

A date spine from nothing

```
1 WITH RECURSIVE dates AS (  
2   SELECT DATE '2026-08-01' AS d  
3   UNION ALL  
4   SELECT d + 1 FROM dates WHERE d < DATE '2026-08-31'  
5 )  
6 SELECT s.store_key, dates.d, COALESCE(a.daily_sales, 0) AS sales  
7   FROM dates  
8   CROSS JOIN dim_store s  
9   LEFT JOIN agg_store_day a  
10      ON a.store_key = s.store_key AND a.sale_date = dates.d;
```

- 8 Cross join to stores, so every store gets every date whether it traded or not.
- 9 Then left join the facts. A closed store gets a row with zero rather than no row at all.

THE TAKEAWAY

This is the fix module six kept pointing at. Once the series is complete, the previous row is the previous day and LAG means what everybody assumed it meant.

When not to generate it

- ◆ **If you have a date dimension, use it.** Kestrel's runs to 2029 and carries the retail calendar the recursion cannot know about.
- ◆ **Postgres has `generate_series`**, which does this in one function call and is clearer than a recursive block.
- ◆ **Most warehouses have a sequence generator** of some kind, under various names, and all of them beat hand-rolled recursion.
- ◆ **Recursion is the portable fallback**, and the right answer when the series is not a simple arithmetic progression.

THE TAKEAWAY

A recursive date spine is a fine answer to an interview question and usually the wrong answer in a warehouse that already has `dim_date`.

Forty Kestrel dashboards each walk `dim_category` recursively to resolve every product's full category path. The tree has 4,912 nodes and six levels, and changes roughly twice a month. The walk takes eleven seconds and runs on every dashboard load. Category paths are also needed by three transformation jobs, each of which has its own copy of the recursive block.

What is the right change?

- ☐ A Cap the recursion at six levels, since the tree is only six deep, which bounds the work.
- ☐ B Rewrite it using `CONNECT BY`, which is the purpose-built hierarchy syntax and is faster than a recursive CTE.
- ☐ C Increase the cluster size, since forty dashboards each walking a tree is a concurrency problem.
- ☐ D Materialise the resolved paths once, nightly, and let every dashboard and job join to that table.

◆ PAUSE AND DECIDE

What is the right change?

A Cap at six levels

Worth doing anyway as a guard, and it does not address the cost. Eleven seconds times forty dashboards is the problem, not the depth.

C Bigger cluster

Pays to repeat the same eleven-second walk forty times an hour. The tree changes twice a month.

B Use CONNECT BY

Oracle-specific, and Kestrel runs Redshift and Postgres. It would also still run on every dashboard load.

D Materialise the paths

A structure changing twice a month, read continuously, is the textbook case for precomputation. Forty-three copies become one.

THE SIGNAL

"Changes roughly twice a month." A hierarchy that is almost static and read constantly should be resolved once and stored. The recursion is right, and its frequency is not.

Rolling a measure up the tree

```
1 WITH RECURSIVE leaf_to_root AS (  
2   SELECT category_key, category_key AS node, 1 AS depth  
3   FROM dim_category  
4   UNION ALL  
5   SELECT l.category_key, c.parent_key, l.depth + 1  
6   FROM leaf_to_root l  
7   JOIN dim_category c ON c.category_key = l.node  
8   WHERE c.parent_key IS NOT NULL)  
9 SELECT r.node AS category_key, SUM(s.net_amount) AS rolled_up  
10  FROM leaf_to_root r  
11  JOIN fact_sales_line s ON s.category_key = r.category_key  
12 GROUP BY r.node;
```

- 2 Each node is seeded as its own ancestor, so a category's own sales are included in its rolled-up total.
- 9 This is a closure table, built on the fly. One row per category and each of its ancestors.

THE TAKEAWAY

Aggregating up a hierarchy is a closure table plus a GROUP BY. Building it in the query is fine once, and scenario three is what happens when forty dashboards do it.

Aggregating over a hierarchy

- ◆ **A node's total is the sum over itself and all its descendants**, which needs the ancestor-descendant pairs, not the parent links.
- ◆ **Summing level by level double counts**, unless you are careful to attribute each fact to exactly one node.
- ◆ **Seeding each node as its own ancestor is what avoids that**, because every fact then joins to exactly one path upward.
- ◆ **And the result is not additive across levels**. Adding a parent's total to its child's counts the child twice.

THE TAKEAWAY

A rolled-up hierarchy total is correct at every level and cannot be summed across levels. Every dashboard that totals a hierarchy column gets this wrong at least once.

A running total is a window. A calculation whose input is **its own previous output** is recursion.

A calculation that windows cannot do

```
1 WITH RECURSIVE bal AS (  
2   SELECT period, opening AS closing, 1 AS n  
3   FROM vw_account_open WHERE period = 1  
4   UNION ALL  
5   SELECT p.period,  
6         b.closing * (1 + p.rate) + p.movement,  
7         b.n + 1  
8   FROM bal b  
9   JOIN vw_account_period p ON p.period = b.period + 1  
10 )  
11 SELECT * FROM bal;
```

- 6 The new closing balance is computed from the previous closing balance. That value exists only because the last pass produced it.

THE TAKEAWAY

No window function can express this. SUM reads input columns, and this expression needs the output of the row before it.

An analyst needs a compounding account balance per period, where each period's closing balance depends on the previous period's closing balance. They write a recursive CTE.

In the recursive term they reference the block inside an aggregate, taking the maximum closing balance found so far. The engine rejects the query. They ask whether recursive CTEs support aggregates.

What should they be told?

- ☐ A Use a window function instead. A running balance is a running total, and SUM OVER an ordered frame expresses it directly.
- ☐ B The recursive term sees only the previous generation and cannot aggregate the self-reference. Carry the running value as a column.
- ☐ C Materialise the chain into a temporary table first, then aggregate over it in a second statement.
- ☐ D Raise the recursion depth limit, since the rejection is a depth guard rather than a syntax restriction.

◆ PAUSE AND DECIDE

What should they be told?

A Use a window function

Right for a running total and wrong here. This balance compounds, so each step's input is the previous step's output, which no window can read.

C Materialise then aggregate

Breaks the dependency the calculation needs. You cannot compute period five's balance before period four's exists.

B Carry it as a column

The working table holds one generation, so there is nothing to aggregate. The previous closing balance rides on the row instead.

D Raise the depth limit

The query was rejected rather than truncated, and depth limits truncate. The restriction is on what the recursive term may contain.

THE SIGNAL

"The maximum closing balance found so far." There is no so far. The working table holds one generation, which is why the aggregate is rejected rather than being wrong.

Reading it in the plan

- ◆ **Look for a Recursive Union node.** That is the loop, and its presence confirms the block is actually recursing.
- ◆ **Underneath it, a WorkTable Scan.** That is the working table being substituted for the self-reference.
- ◆ **Check the actual row count on the Recursive Union.** Far more than your tree has nodes means you are going round.
- ◆ **Index the join column.** A recursive walk is one lookup per node per pass, and an unindexed parent key makes it a scan.

THE TAKEAWAY

The row count on the Recursive Union node is the fastest cycle detector there is. Four thousand nodes and two million rows is not a deep tree.

The result grows with the tree. The **working table** grows with the widest level.

Estimating the cost

- ◆ **Passes equal depth.** Six levels means six passes, plus one that returns nothing and ends the loop.
- ◆ **Each pass is one join,** from the working table to the source table. Index the column it joins on.
- ◆ **Nothing parallelises across passes,** because pass four cannot start until pass three has finished.
- ◆ **Within a pass it parallelises normally,** which is why a wide shallow tree behaves far better than a narrow deep one.

THE TAKEAWAY

Depth is latency and width is throughput. A tree that is six deep and two thousand wide is a good shape. One that is two thousand deep is not.

A depth cap, written defensively

```
1  UNION ALL
2  SELECT c.category_key, c.parent_key, w.depth + 1,
3         w.path || c.category_key
4  FROM dim_category c
5  JOIN walk w ON c.parent_key = w.category_key
6  WHERE w.depth < 20           -- the tree is 6 deep
7         AND NOT c.category_key = ANY(w.path)
```

- 6 A cap well above the real depth. It never fires on good data and it bounds the damage on bad data.
- 7 And the path check, which is module twelve. Do not revisit a node already on this row's route.

THE TAKEAWAY

A cap of twenty on a six-deep tree costs nothing and turns an unbounded query into a bounded one. Write it even when you are sure the data is a tree.

Three guards worth writing

- ◆ **A depth cap, above the real depth.** One comparison per row, and it bounds every failure mode at once.
- ◆ **A path check,** so a node already on this row's route is not visited again. Module twelve goes into this properly.
- ◆ **A row-count assertion afterwards,** comparing the walk's output against the node count you expected.
- ◆ **None of them is default behaviour** on Postgres, Redshift or Snowflake. All three are yours to remember.

THE TAKEAWAY

SQL Server's hundred-level default cap is the only one of these any engine gives you, and it is the one that also breaks legitimate deep hierarchies.

Asserting the walk

```
1 SELECT (SELECT COUNT(*) FROM walk)          AS rows_walked,  
2       (SELECT COUNT(*) FROM dim_category) AS nodes_in_tree,  
3       (SELECT MAX(depth) FROM walk)         AS deepest,  
4       (SELECT COUNT(*) FROM walk  
5        GROUP BY category_key  
6        HAVING COUNT(*) > 1 LIMIT 1)        AS any_node_twice;
```

- 1 For a tree, rows walked equals nodes reached. More rows than nodes means some node was visited twice.
- 6 And this returns nothing at all on a clean tree. Any row here is a node reached by two different routes.

THE TAKEAWAY

Two numbers and a probe. On a tree these are boring every time, and on the day they are not, you have found the cycle before it found you.

Testing a hierarchy walk

- ◆ **Build a fixture with a known shape.** Three levels, one node with two children, one leaf. Count the expected rows by hand.
- ◆ **Then add a cycle to it deliberately,** and assert that your guarded query terminates and flags it.
- ◆ **Production data will not test the cycle case,** because it either has one and hangs, or does not and proves nothing.
- ◆ **Assert the depth, not just the row count.** A walk that stops one level short returns fewer rows and no error.

THE TAKEAWAY

The second point is the one that matters. A cycle in a fixture is the only way to find out what your query does about a cycle before the data provides one.

Writing one, in four decisions

01

Seed

What starts it? Roots, or a single node, or a constant. Filter hard here.

02

Step

How do you get from one row to the next? That is the join in the recursive term.

03

Carry

What does each row need to know? Depth, path, running value. Decide before writing.

04

Stop

What makes the working table empty? Write it down, and add a depth cap regardless.

THE TAKEAWAY

Step four is the one people leave implicit. A stopping condition that depends on the data being clean is not a stopping condition.

The confusions, settled

SYMPTOM	THE CAUSE
Value too long, at a fixed depth	The anchor fixed the column type. Cast it there.
The walk returns only the seed rows	The join in the recursive term is the wrong way round.
UNION does not stop a loop	A depth or path column makes every row unique.
Aggregate rejected in the recursive term	The working table holds one generation. Carry it as a column.
Rows come back level by level	That is breadth-first, which the algorithm produces. Order by a path.
Far more rows than the tree has nodes	Something is being visited repeatedly. Module twelve.

THE TAKEAWAY

The second row is the most common by a distance, and the easiest to miss, because a query returning only the roots looks like a query returning nothing much.

Five habits

- ◆ **Carry depth and path on every walk**, whether or not you need them yet. Both are hard to add afterwards.
- ◆ **Cap the depth well above the real depth**. It never fires on good data and it bounds every failure.
- ◆ **Filter the anchor as hard as you can**. Everything downstream is proportional to the seed.
- ◆ **Use UNION ALL, and handle cycles explicitly**. UNION is a cost, not a guard, once you carry a depth column.
- ◆ **Cast the anchor's columns deliberately**. They are the types of the whole result, including the ones that grow.

THE TAKEAWAY

Four of these cost one line each. The fifth costs nothing and prevents the failure in scenario one, which is the one that looks like a data problem.

A footnote: WITH ITERATIVE

- ◆ **There is a live proposal to add an iterative variant** to PostgreSQL, distinct from the recursive one.
- ◆ **Recursive CTEs append.** Every generation is added to a result that only grows, which is wrong for some algorithms.
- ◆ **The proposed form replaces instead**, keeping only the latest generation, and stops on a user-defined predicate.
- ◆ **Which is what PageRank and clustering need**, where a fixed set of rows has its values updated repeatedly.

THE TAKEAWAY

Worth knowing because it names the boundary. Recursive CTEs are for growing a relation. Using them to update one repeatedly is a misuse that happens to work.

Everything here assumed the data is a tree.
Module twelve is what happens when it is not.